

---

# Typed Planning and Deterministic Execution for Verifiable Public-Procurement Question Answering

Student Name\*

Department and University

student@example.ac.uk

## Abstract

Public-procurement data can support questions about buyers, suppliers, contract notices, and disclosure, but many useful questions require exact set construction, multi-hop joins, aggregation, and comparison rather than retrieval of a few relevant passages. We present a question-answering pipeline that maps natural-language questions to programs in a closed, typed algebra and delegates factual authority to deterministic execution over a provenance-preserving procurement knowledge graph. Training programs are generated from the data and retained only after type checking, non-degeneracy checks, and agreement between two execution implementations. For deployment, a structured understander is followed by either a deterministic intent compiler or, when the intent cannot be compiled safely, a typed graph planner; grounding, exhaustive execution, evidence checks, and bounded replanning then produce an answer or a controlled non-answer. On the frozen 922-item PACS compositional test, an 8B parameter model trained with typed programs obtains 74.51% strict execution accuracy on the canonical surface and 73.21% on an independently naturalised surface, compared with 36.01% for the same untuned base model and 48.81% for a cloud-model planner. On a separate balanced 260-item end-to-end snapshot, the full local runtime scores 86.15%, compared with 70.38% for its untuned counterpart. The results support typed execution as a practical way to improve compositional reliability, while audits also expose unresolved limitations in currency normalisation, entity aliases, benchmark isolation, and artifact portability.

## 1 Introduction

Public procurement records describe how public buyers solicit work, select suppliers, and publish awards and contracts. The Open Contracting Data Standard (OCDS) provides a common release and record model for these disclosures, including parties, tenders, awards, contracts, and linked documents ([Open Contracting Partnership, 2026](#)). The resulting data can support questions such as which suppliers serve multiple buyers, whether disclosure is complete for a class of notices, or how activity changes across years. These are not, however, ordinary document lookup questions. Their answers may depend on the role of an organisation in one record, a set of organisations derived from that record, a second retrieval bound by the derived set, and a final set, count, comparison, or ranking operation.

This distinction matters because retrieving *relevant* evidence is not equivalent to retrieving the *complete* denotation required by an aggregate. Let  $R(q)$  be the complete set of contract records satisfying question  $q$ , and let a top- $k$  retriever return  $\hat{R}(q) \subset R(q)$ . Even when  $\hat{R}(q)$  contains convincing examples, it does not in general follow that

$$|\hat{R}(q)| = |R(q)|, \quad \sum_{r \in \hat{R}(q)} v(r) = \sum_{r \in R(q)} v(r), \quad (1)$$

---

\*This draft was prepared with language-model assistance for literature search, organisation, and language editing. The author is responsible for checking every claim, experiment, citation, and final wording.

---

**Author-drawn Figure 1: visual thesis and running example.**

Show one real procurement bridge question, the structured intent produced by the understander, the actually taken compiler/planner branch, the source-record set, emitted bridge values, bound target-record set, deterministic result, and release checks. Every displayed value must come from one saved end-to-end trace; unused branches are grey.

Figure 1: Intended content for the introduction figure. The final artwork will instantiate a single saved end-to-end trace rather than combine artifacts from the separate compositional evaluation protocol. Runtime verification denotes structural, execution, and evidence checks; oracle equality is an offline evaluation criterion.

or that a ranking computed from the retrieved subset has the same maximum. Retrieval-centred KGQA systems are effective at finding answer-bearing subgraphs (Sun et al., 2019; Jiang et al., 2023b; Luo et al., 2024), but exact counts and compositional set operations additionally need explicit scope, typed intermediate values, and exhaustive execution within the declared data universe.

Direct language-model generation introduces a different failure mode. A model can produce a well-formed query that reverses buyer and supplier roles, projects a contract identifier when the question requests an organisation name, or aggregates a non-additive monetary field. Such answers can remain fluent and superficially plausible. Prior work therefore represents complex questions with executable programs (Cao et al., 2022; Wolfson et al., 2020), constrains formal generation (Scholak et al., 2021; Shu et al., 2022), or uses execution and environment feedback (Chen et al., 2021; Huang et al., 2024). These ideas motivate our central design choice: the language model proposes a typed computation, while the data system—not the model’s prose—has final authority over factual values.

We implement this idea at two connected levels. First, a compact model is trained to translate procurement questions into a closed algebra containing record filters, distinct projections, semi- and anti-joins, set operations, grouping, aggregation, comparison, and ranking. Every program is type checked and executed deterministically over a frozen record universe. Second, the deployment runtime adds a structured understanding stage and a conditional planner. A valid intent is compiled deterministically when possible; otherwise a graph planner produces a typed variable-dependency plan. The plan is grounded against allowed schema fields, executed exhaustively, and passed through preflight, postflight, answer-sanity, and evidence checks. Structured defects may trigger bounded replanning, but a clean empty result is terminal so that the system cannot search repeatedly until it finds a preferred answer.

The distinction between the two levels is methodological rather than cosmetic. Our primary compositional evaluation asks whether a single model call emits a valid algebra tree or an appropriate abstention, after which a deterministic evaluator computes the answer and an offline scorer compares it with a frozen oracle. It does not invoke the deployment understander, graph planner, evidence bundle, or repair loop. A separate system evaluation measures the complete runtime. Keeping these protocols separate prevents an offline oracle check from being mislabeled as an inference-time verifier and prevents a component that was not called for a test item from receiving credit for that item.

Experiments use a provenance-preserving graph constructed from UK procurement releases from 2022–2026 and a frozen 922-question compositional test. The typed-program model attains 74.51% strict accuracy on the canonical question surface, a gain of 38.50 percentage points over the same untuned 8B base model and 25.70 points over a cloud planner. Accuracy is 73.21% after independent surface naturalisation, a drop of only 1.30 points. The strongest gains occur on overlap/exclusion and relational composition, whereas supplier-concentration and disclosure/compliance remain substantially harder. A separate balanced 260-item

---

artifact evaluates the complete runtime and records 86.15% accuracy, compared with 70.38% for the untuned system. We treat the latter as secondary evidence because its stored outputs are less portable than the PACS artifacts.

The contributions of this work are therefore:

- a provenance-preserving procurement graph and a deterministic query-record projection, with explicit entity-resolution, additivity, and coverage checks;
- a closed typed algebra and program-first supervision pipeline for exact set, join, aggregation, comparison, and ranking questions;
- a full question-answering runtime that separates learned understanding and planning from grounding, exhaustive execution, evidence checks, bounded repair, and controlled non-answering; and
- PACS, a frozen compositional evaluation with two surface channels, operator-family and depth diagnostics, answerability controls, and an audit of the limitations of its isolation and scoring procedures.

The contribution is deliberately narrower than open-domain question answering. The pipeline does not guarantee that the source data are complete in the world, that stored organisation names are unique legal entities, or that passing an execution check proves semantic correctness. It instead makes a more testable claim: within a declared frozen data universe, the model’s proposed computation can be type checked, executed, traced, and rejected when its implemented preconditions fail.

## 2 Related Work

### 2.1 Executable semantic parsing

Neural-symbolic question answering separates a learned programmer from a symbolic executor, making the answer a denotation of an explicit program rather than unconstrained generated text (Liang et al., 2017). KQA Pro develops this idea into KoPL, a compositional language whose programs expose multi-hop, qualifier, comparison, logical, and counting operations (Cao et al., 2022). Question decomposition similarly represents a complex utterance as smaller executable or answerable units (Wolfson et al., 2020). Our algebra follows the same high-level principle but is specialised to a contract-record universe: it distinguishes record sets from sets of scalar field values, and includes semi-joins, anti-joins, set difference, grouped aggregation, and deterministic ranking.

Formal output alone does not make generation reliable. PICARD rejects inadmissible SQL tokens during decoding (Scholak et al., 2021); TIARA combines multi-grained KB retrieval with constrained logical-form generation (Shu et al., 2022); and ReTraCk applies syntactic, ontology, real-execution, and virtual-execution checks in a modular KBQA framework (Chen et al., 2021). Our implementation does not perform PICARD-style token-level constrained decoding. It instead validates the completed typed object, compiles and grounds it, and applies execution- and evidence-stage checks. This difference is important because each mechanism rejects a different class of errors.

Execution is also an imperfect correctness signal. Incorrect programs often execute without a runtime error, and executability has accordingly been treated as weak supervision rather than a semantic certificate (Wang et al., 2021). Execution-guided refinement can improve complex KBQA (Guo et al., 2023), while QueryAgent uses environmental feedback to correct action sequences (Huang et al., 2024). Our bounded repair uses structured failures to request a new plan and then repeats grounding, execution, and verification. It never permits a language model to replace the executor’s factual answer.

### 2.2 Retrieval and reasoning over structured data

Graph-based QA methods retrieve or construct question-specific subgraphs before selecting an answer. GRAFT-Net fuses knowledge-base and text nodes in a heterogeneous graph (Sun et al., 2018); PullNet learns

which nodes to expand during iterative retrieval (Sun et al., 2019); UniKGQA unifies retrieval and reasoning with a shared matching function (Jiang et al., 2023b); and RoG uses an LLM-generated relation path to retrieve faithful reasoning paths (Luo et al., 2024). StructGPT instead exposes specialised interfaces through which an LLM incrementally reads structured data (Jiang et al., 2023a). These systems show the value of schema and graph structure, but their core objective is usually answer-entity or answer-bearing-path retrieval. For exact procurement counts, set differences, and group comparisons, our objective is the complete denotation of an explicitly scoped program within a frozen backend, not a small relevant subgraph.

Table-reasoning work provides a complementary perspective. Chain-of-Table makes intermediate tables part of the reasoning state (Wang et al., 2024), illustrating why visible state transformations can be more auditable than free-form chains of thought. Our execution trace similarly records typed intermediate sets, but operations are evaluated by code over a fixed record universe rather than by an LLM that edits a displayed table.

## 2.3 Compositional generalisation and answerability

Compositional benchmarks ask whether known primitives can be recombined in unseen structures. CFQ formalises this through atom and compound divergence (Keysers et al., 2020); COGS separates lexical and structural generalisation cases (Kim & Linzen, 2020); and semantic-parsing studies show that improvements on out-of-distribution compositions can coexist with a large remaining gap (Oren et al., 2020). Hybrid systems can use a high-precision grammar path and fall back to a neural parser when the grammar does not cover an utterance (Shaw et al., 2021). This is closely related to our conditional routing from a deterministic intent compiler to a learned graph planner. We do not claim the formal maximum-compound-divergence guarantee of CFQ: PACS controls exact questions, tree hashes, and declared unseen shapes, while its near-duplicate check is approximate (Section 5.2).

Real KBs also make some questions unanswerable. GrailQAbility distinguishes missing schema, entity, and fact conditions and evaluates both answerable and unanswerable cases (Patidar et al., 2023). Our evaluation similarly includes unsupported operations and clean no-result cases, but the semantics are local to the implemented algebra and frozen procurement graph. A no-result outcome can be the correct denotation; it is not treated as an invitation to search for a different answer.

## 2.4 Public-procurement knowledge graphs

The TheyBuyForYou platform demonstrates how procurement releases, company data, ontologies, entity reconciliation, and document services can be integrated into a public-procurement knowledge graph (Soylu et al., 2022). Our work shares the data-integration motivation but studies a different research question: how a language model can propose a typed, auditable computation whose denotation is obtained from such structured data. The graph itself is therefore infrastructure and an evaluated data contribution, not evidence that the QA method is correct.

# 3 Problem Setup and Guarantee Boundary

## 3.1 Data universe

The input is a finite snapshot of OCDS releases  $D$ . A deterministic data pipeline transforms  $D$  into a knowledge graph  $G = (V, E)$  and a query-record projection  $U_G$ . Each row  $r \in U_G$  is keyed by a contract-node identifier and exposes a controlled set of fields, including stored buyer and supplier names, release year, procurement category and CPV code, selected dates, a monetary value and currency when available, an additivity flag, and provenance links. The word *record* in the remainder of the paper refers to one row of  $U_G$ , not to an arbitrary text passage or to every versioned OCDS release.

The projection deliberately preserves two different notions of identity. Graph organisation nodes use canonical identifiers produced by entity resolution. Query fields such as `buyer_name` and `supplier_name`, however, are stored strings derived from the first-party data. A set of distinct stored names must therefore not be

interpreted as a set of perfectly resolved legal entities. We retain this distinction in result descriptions and qualitative examples.

### 3.2 Questions, programs, and denotations

Let  $q$  be a natural-language question,  $\mathcal{L}$  the closed typed algebra in Section 4.3, and  $p \in \mathcal{L}$  a program. Static validation assigns a root type  $\tau$  to a program,

$$\Gamma \vdash p : \tau, \quad \tau \in \{\text{Records, Values, Groups, Number, Value, Bool, Ranking}\}, \quad (2)$$

or returns a structured type error. Deterministic evaluation gives an answer envelope

$$E_G(p) = (s, \tau, y), \quad (3)$$

where  $s$  is an execution status and  $y$  is the denotation when  $s = \text{OK}$ . A surface renderer may verbalise  $y$ , but it may not introduce a factual value not contained in  $y$ .

The full runtime uses a richer intermediate representation. A structured intent  $i$  describes the requested answer signature, source constraints, derived entity sets, target constraints, and return operation. A compiler or conditional planner maps  $i$  to a graph plan  $H = (X, A)$ , where each variable  $x \in X$  is a typed record or entity set and each directed edge in  $A$  denotes a dependency whose output is bound into a downstream constraint. Compilation produces executable low-level specifications, which are grounded and evaluated over  $U_G$ .

### 3.3 Answerability

Each evaluation item has one of three operational statuses:

**Answerable.** A program in  $\mathcal{L}$  can express the requested computation and its implemented data preconditions are present in the frozen snapshot.

**Unsupported.** The question requests an operation outside  $\mathcal{L}$ , such as a median or mean in the current benchmark grammar. Correct behaviour is an explicit abstention.

**No result.** The requested, well-formed scope has an empty denotation in  $U_G$ . Depending on the answer signature, correct behaviour is a faithful empty/zero result or an explicit no-result response, not a fabricated alternative.

Ambiguity and insufficient evidence are additionally represented in the full runtime. They are not collapsed into the same state: an unsupported operator, a valid empty query, an unresolved literal, and a missing evidence precondition require different user-facing explanations.

### 3.4 Five non-equivalent notions of success

Table 1 defines the terminology used throughout the paper. The distinctions prevent a common but consequential inference: a program can be type valid and execute successfully while still expressing the wrong question.

For an answerable compositional item, strict correctness is

$$\mathbb{K}_{\text{strict}} = \mathbb{K}[\Gamma \vdash p : \tau] \mathbb{K}[E_G(p).s = \text{OK}] \mathbb{K}[\text{match}_\tau(E_G(p).y, y^*)], \quad (4)$$

where  $y^*$  is the frozen oracle. Set-valued answers are compared as sets of strings, rankings preserve their defined order and tie semantics, Booleans require Boolean values on both sides, and numeric values use the evaluator’s documented tolerance. In particular, Python’s subtyping of `bool` under `int` must not make `True` equivalent to numeric one. Unsupported cases are correct only when the model explicitly declines the operation; no-result cases use their declared empty-result convention.

Table 1: Guarantee boundary. Only oracle match is an offline answer-correctness measurement; none of the first four rows alone establishes real-world truth.

| Term               | Meaning in this paper                                                                                                        |
|--------------------|------------------------------------------------------------------------------------------------------------------------------|
| Static validity    | The object belongs to the closed grammar and all operator input/output types compose; depth and node limits are respected.   |
| Executability      | The grounded object can be evaluated by the implemented backend without a structural or runtime failure.                     |
| Runtime verified   | The implemented compile, preflight, execution, postflight, sanity, and evidence conditions required for release have passed. |
| Evidence supported | The answer is associated with the record/evidence identifiers emitted by execution under the implemented provenance policy.  |
| Oracle match       | The typed denotation equals the frozen benchmark oracle under the stated exact, set, Boolean, or numeric comparison rule.    |
| Real-world truth   | A claim about procurement activity outside the finite snapshot; this is not guaranteed by the system or benchmark.           |

Runtime verification is measured independently from Equation 4. When both labels are available, the informative diagnostic is the  $2 \times 2$  table of verifier acceptance versus oracle correctness: it reveals both accepted errors and rejected correct plans. The present artifacts do not support every cell of this analysis for every experiment, so we report it only where trace provenance is complete rather than inferring it from final accuracy.

## 4 Method

### 4.1 Overview: two connected execution paths

The system contains a shared deterministic data substrate and two deliberately distinct language-model interfaces.

The **typed-program path** maps a question directly to either an algebra tree or an abstention object. It is used to generate program supervision and to measure compositional semantic-parsing capability. A tree is validated, executed over the query-record universe, and scored against an offline oracle. This path does not create an evidence bundle or call the deployment repair loop.

The **full runtime** first elicits a structured intent. A router then chooses a deterministic compiler whenever the intent can be compiled without either of two implemented fast-path vetoes; remaining questions are sent to a typed graph planner. Both branches converge on a graph/low-level specification that is grounded, preflighted, executed, and checked before an answer is released. A bounded feedback path is available only for eligible structured defects. Figure 2 is reserved for this general control flow, whereas Figure 1 must contain one concrete trace.

### 4.2 From OCDS releases to a queryable graph

#### 4.2.1 Streaming ingestion and release selection

The ingestion stage scans yearly gzip-compressed JSON Lines files. It skips empty and malformed lines rather than terminating a multi-year build, unwraps release packages, and flattens the fields needed by downstream extraction. OCDS publishes a new release when information about a contracting process changes ([Open Contracting Partnership, 2026](#)); consequently, multiple rows can share an Open Contracting ID (OCID). For each OCID, the implementation retains the row with the latest parseable release date and records all observed source years. This policy creates a current snapshot rather than a complete event history. It is deterministic for a fixed input ordering and explicitly differs from merging all release versions field by field.

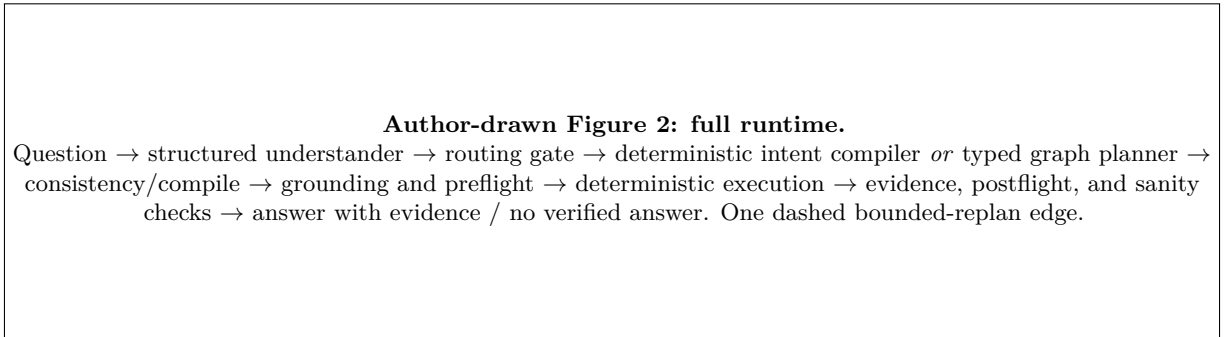


Figure 2: General full-runtime control flow. A directly compilable intent bypasses the graph model; a structured failure may trigger bounded replanning, after which the complete ground–execute–check sequence is repeated. The separate typed-program evaluation omits these deployment stages.

The full pipeline entry point accepts an optional year subset. In the current implementation, partial runs must be directed to an explicit non-canonical output path; this prevents a small diagnostic run from silently replacing a complete artifact. The canonical full build preserves its original default path.

#### 4.2.2 Role-aware entity resolution

Entity resolution is conservative because a false merge affects every downstream join. Phase 1 uses official organisation identifiers when an observed identifier belongs to an accepted official scheme. A publication-system identifier (such as a GB-FTS identifier) can become an alias only when observations with the same OCID and normalised name share a party role and the complete candidate set points to exactly one official canonical identifier. Candidate identifiers are accumulated across observations before the uniqueness decision; a first-row match is not sufficient. This rule avoids merging two same-named parties that occur as buyer and supplier or that map to different official organisations.

Phase 2 resolves remaining parties in three deterministic passes: a government lookup table; exact normalised name plus region; and, when region is absent, exact normalised name. A resolved cluster receives a stable identifier derived from SHA-256. An observation whose name normalises to an empty string is retained as its own singleton instead of disappearing from the output. Jaro–Winkler similarity at a threshold of 0.92 is used only to emit review candidates; it never performs an automatic fuzzy merge. Thus the resolver favours false negatives over irreversible false positives.

#### 4.2.3 Fact extraction and provenance

The extraction stage produces normalised tables for tenders, lots, award criteria, awards, bid statistics, text evidence, and documents. For an award-level monetary value, the implementation uses the award value when present, otherwise a linked contract value, and finally a tender value as a fallback. Only award- and contract-sourced values are labelled `value_is_additive`; tender fallbacks are not. A numeric zero is a valid value and is not replaced by a later fallback. Currency is stored but not normalised.

Every extracted fact retains identifiers that link it to the OCID, release, award or contract, and source evidence where available. These links permit the runtime to return record-level provenance, but they do not prove that a publisher supplied every real-world fact.

#### 4.2.4 Graph construction and validation

The graph contains organisation, contract/notice, CPV, and evidence nodes. Buyer and supplier edges connect canonical organisations to contract nodes; category edges connect contracts to CPV nodes; and evidence edges connect source artifacts to contracts. Contract-node identifiers are deterministically formed from the OCID and award identifier. Before publication, the build checks primary-key uniqueness, referential integrity, JSON serialisability, additivity labels, and coverage thresholds. The frozen experimental graph



Table 2: Core algebra operators.  $P$  is a predicate,  $f$  a field,  $m$  an aggregate metric, and  $\bowtie_s$  an expression-valued membership predicate.

| Family      | Signature                                                                 | Semantics                                |
|-------------|---------------------------------------------------------------------------|------------------------------------------|
| Filter      | $\text{filter}_P : \text{Records} \rightarrow \text{Records}$             | Select records                           |
| Project     | $\text{values}_f : \text{Records} \rightarrow \text{Values}$              | Distinct non-empty values                |
| Cardinality | $\text{count} : \text{Records} \rightarrow \text{Number}$                 | Distinct contract count                  |
|             | $\text{size} : \text{Values} \rightarrow \text{Number}$                   | Set cardinality                          |
| Aggregate   | $\text{sum}_f : \text{Records} \rightarrow \text{Number}$                 | Decimal accumulation                     |
| Group       | $\text{groupby}_{f,m} : \text{Records} \rightarrow \text{Groups}$         | Keyed count or sum                       |
| Set         | $\text{union/intersect/diff} : \text{Values}^2 \rightarrow \text{Values}$ | Set algebra                              |
| Bridge      | $\text{filter}_{f \bowtie_s e}$                                           | Semi-/anti-join with $e : \text{Values}$ |
| Rank        | $\text{argext} : \text{Groups} \rightarrow \text{Value}$                  | Extreme group key                        |
|             | $\text{top}_k : \text{Groups} \rightarrow \text{Ranking}$                 | Ordered key-value pairs                  |
| Scalar      | $\text{combine} : \text{Number}^2 \rightarrow \text{Number/Bool}$         | Arithmetic/comparison                    |
| Boolean     | $\text{exists} : \text{Records} \rightarrow \text{Bool}$                  | Non-empty test                           |

contains 215,221 contract nodes, 131,502 organisation nodes, 3,870 CPV nodes, and 535,731 evidence nodes. Exact edge and coverage counts appear in Table 3.

The query backend materialises a flat record universe  $U_G$  from these Parquet nodes and edges. This representation intentionally makes the unit of **count** explicit: duplicate rows are deduplicated by **contract\_node\_id**. Buyer/supplier fields use the corresponding stored name projection. Grouping drops null and empty keys. This record view is the contract between the algebra and the graph; changing it changes benchmark semantics and therefore requires a new snapshot hash.

### 4.3 Closed typed algebra

The algebra is *closed* in its operator inventory but permits open composition of those operators. Static validation limits a tree to depth 16 and 64 nodes and returns one of seven root types. Table 2 summarises the principal signatures; Appendix B provides the complete inventory and failure semantics.

Predicates support equality, membership, containment, existence, bounded numeric comparison, negation, disjunction, and expression-valued membership. The last form is the key to a bridge query. For example, if  $k$  is an anchor buyer name,

$$S = \text{values}_{\text{supplier\_name}}(\text{filter}_{\text{buyer\_name}=k}(U_G)), \quad (5)$$

$$B = \text{values}_{\text{buyer\_name}}(\text{filter}_{\text{supplier\_name} \in S}(U_G)), \quad (6)$$

$$A = B \setminus \text{values}_{\text{buyer\_name}}(\text{filter}_{\text{buyer\_name}=k}(U_G)). \quad (7)$$

The types make two common errors visible before execution: filtering **supplier\_name** by a buyer anchor is a role error detectable only from semantics, but returning contract identifiers where a set of buyer values is expected also violates the declared answer signature.

Static validation checks tree shape and type composition, not the truth of the program or every domain guard. Field existence is checked during evaluation. Monetary training templates add an explicit **value\_is\_additive=true** predicate, but the generic **sum** operator does not itself prove that this predicate is present, and neither evaluator currently enforces a single currency. We therefore treat monetary results as subject to the limitation in Section 8.

### 4.4 Deterministic execution

The runtime algebra evaluator first calls the static validator, then evaluates the tree recursively over  $U_G$ . A record filter constructs a Boolean mask. An **in\_expr** predicate first evaluates its child to a distinct value set and then applies membership to the parent field. Counts deduplicate contract identifiers. Distinct projection removes null and empty strings. Numeric sums deduplicate records and accumulate with **Decimal**;



---

non-finite or non-numeric values fail the execution. Grouped counts deduplicate (group key, contract id) pairs. Rankings use a deterministic value-and-key ordering, and select operations fail rather than guess when more than one distinct answer remains.

A second evaluator is implemented separately for dataset generation and audit. A candidate training instance is retained only when the system evaluator and this independent implementation both return successful envelopes and their answers agree under the generator comparator. This reduces circularity between template generation and execution, although it is not formal verification: the two implementations can share assumptions, and the historical comparator had an overly permissive Boolean coercion rule (Section 8).

#### 4.5 Program-first supervision

Training examples are sampled from the actual record distribution rather than generated as unanchored text. A sampler selects concrete buyers, suppliers, years, CPV codes, categories, and other field values; instantiates a typed program; renders a natural-language instruction; and executes the program. We reject a sample if (i) the tree fails static validation, (ii) either evaluator fails, (iii) the evaluators disagree, (iv) the answer is degenerate for its declared type, or (v) the question string has already been accepted. Degeneracy rules exclude zero-valued numeric examples, empty value sets, singleton rankings, and missing scalar answers. These rules are a data-construction heuristic, not a statement that zero is intrinsically invalid at inference time.

The resulting LLaMA-Factory export contains 12,414 training and 253 validation examples. Each target is JSON containing either a typed tree or, for unsupported evaluation controls, an abstention object. The specialised parser uses Qwen3-8B (Yang et al., 2025) with 4-bit QLoRA (Dettmers et al., 2023). LoRA rank is 64, scaling  $\alpha = 128$ , dropout is 0.05, and all target modules are adapted. Training runs for two epochs with per-device batch size 2, gradient accumulation 8 (effective batch 16), learning rate  $10^{-4}$ , cosine decay, 3% warm-up, a 4096-token cutoff, and bfloat16 computation. The held-out synthetic loss is reported only as an optimisation diagnostic because the export contains duplicates and two exact train/validation overlaps.

#### 4.6 Schema-grounded understanding and conditional planning

The full runtime begins by retrieving a compact schema context and requesting a structured intent program. The intent records the question shape, answer signature, source record constraints, intermediate entity-set roles, target constraints, and return operation. Provider-enforced JSON schema output is used when available, with a text/JSON fallback for older clients. A malformed briefing receives one cheap structural retry; an unparseable result becomes a typed planning failure rather than a factual answer.

If the object is a recognised intent program, a deterministic compiler maps it to a graph plan. The compiled fast path is accepted unless compilation fails or one of two narrow semantic vetoes fires: an unsupported cue that could be converted into an invented answer, or a mismatch between an entity-set count and a record-count question. The implementation intentionally does not run the raw-planner consistency checker over every compiled graph, because checks calibrated for raw model payloads produced false positives on deterministic compiler output.

Questions that do not pass this gate invoke the Step-2 graph planner. The planner emits typed variables, dependencies, operation units, and a return specification. A consistency checker tests the payload against the question, and the compiler normalises and attaches low-level query specifications. Structural compile/consistency failures may trigger a small number of fresh plan samples. An executable but semantically wrong plan is not detectable from structure alone and is not resampled using an oracle at inference time.

This conditional design is related to high-precision grammar plus neural fallback systems (Shaw et al., 2021). A complete routing ablation would compare compiler-only, always-planner, and conditional modes under the same model and budget. The present artifact set does not contain that complete three-way experiment, so the paper describes the implemented mechanism but does not attribute the entire end-to-end gain to routing.

---

**Algorithm 1: Full question-answering runtime**

1. Obtain structured intent  $i \leftarrow \text{UNDERSTAND}(q, \text{schema})$ .
2. If  $i$  compiles and no fast-path veto fires, set  $H \leftarrow \text{COMPILE}(i)$ ; otherwise obtain and consistency-check  $H \leftarrow \text{PLAN}(q, i)$ .
3. For each attempt up to the fixed budget: ground all variables and literals; run preflight; execute variables in dependency order; construct evidence; run postflight, sanity, and evidence checks.
4. If all release conditions pass, return an answer card whose factual value is the executor denotation.
5. If the defect is repair-eligible, send structured feedback to the planner and return to Step 3. Otherwise return the corresponding unsupported, ambiguous, no-result, or insufficient-evidence non-answer.

Figure 3: Runtime control procedure. Repair changes the plan, never the factual answer directly, and repeats the complete deterministic path.

#### 4.7 Grounding and graph execution

Grounding maps aliases to canonical backend fields, normalises values, resolves confident organisation mentions, and rejects fields outside the allowed backend profile. It also attaches operation-specific guards such as the additivity predicate used for monetary operations. A preflight phase checks schema support and query preconditions before any answer is produced.

For a graph plan, variables are executed in dependency order. A source record-set variable applies its grounded constraints. An entity-set variable emits distinct values from its source. The emitted set is then bound to the appropriate field of a dependent target variable—for example, supplier values are bound to `supplier_name`, whereas a record-set intersection binds contract identifiers. Variables at the same dependency level may execute in parallel; a terminal operation computes the requested count, set, comparison, or other supported answer. Each variable trace stores its status, grounded constraints, emitted field, output size, and answer summary. The executor is the sole factual authority: the planner is never asked to invent the terminal value.

#### 4.8 Evidence, release checks, and answer cards

Execution returns record and evidence identifiers in an `EvidenceBundle`. Postflight checks inspect operation-specific properties of the returned rows, while answer-sanity checks detect incompatible shapes, non-finite numbers, and other release defects. An evidence verdict summarises whether the implemented support requirement is satisfied. A released `AnswerCard` contains the denotation or its verbalisation, confidence label, evidence identifiers, and limitations. If a required check fails, the runtime produces no verified answer.

These checks provide operational guarantees, not an oracle. They can establish that a program uses allowed fields, executes over the declared scope, and returns linked records. They cannot establish that the plan captures the user’s intended semantics or that the publisher’s data are complete. We therefore reserve *correct* for the offline oracle metric and use *runtime verified* only in the sense defined by Table 1.

#### 4.9 Bounded replanning and abstention

Algorithm 3 summarises the implemented control logic. Repair is eligible when a structured defect supplies actionable feedback: compilation or consistency rejection, failed preflight/postflight checks, an execution/verifier failure, or a no-result plan containing a literal not traceable to the question. The feedback object records the failure stage, failed checks, grounding information, and invented literals. A new plan is then grounded, executed, and checked from the beginning. No repaired answer is accepted merely because its surface form looks better.

Conversely, if a fully grounded query executes to no results and every literal is traceable to the question, the empty result is terminal. Replanning in this case would amount to answer-shopping. Unsupported operations, unresolved ambiguity, exhausted repair budgets, and insufficient evidence all terminate with a controlled non-answer.

Table 3: Frozen graph and PACS evaluation statistics. PACS family counts are intentionally non-uniform, so overall accuracy is a micro average and macro-family accuracy is also reported.

| Block             | Item                                 | Count           |
|-------------------|--------------------------------------|-----------------|
| Graph nodes       | Contract/notice                      | 215,221         |
|                   | Organisation                         | 131,502         |
|                   | CPV                                  | 3,870           |
|                   | Evidence                             | 535,731         |
| Contract coverage | Buyer edge                           | 215,218         |
|                   | Supplier edge                        | 204,186         |
|                   | Category edge                        | 164,691         |
|                   | At least one evidence edge           | 215,202         |
| PACS              | Each surface channel                 | 922             |
|                   | Answerable / unsupported / no result | 694 / 138 / 90  |
|                   | Level L1 / L2 / L3                   | 279 / 339 / 304 |
|                   | Seen / declared unseen shapes        | 431 / 491       |
|                   | Smallest family (F7)                 | 67              |
|                   | Largest families (F2 and F6)         | 149 each        |

## 5 Experimental Setup

The experiments address four questions:

**RQ1:** Does typed-program supervision improve strict compositional execution accuracy over the same untuned base model and a strong cloud planner?

**RQ2:** Does the specialised parser retain its accuracy under an independently generated surface form?

**RQ3:** Which operator families and composition depths remain difficult?

**RQ4:** Does the complete understand-plan-ground-execute-check runtime improve over its untuned counterpart on a balanced system snapshot?

### 5.1 Frozen graph snapshot

All reported executions use the same Parquet knowledge-graph snapshot and query-record projection. Table 3 reports graph size and contract-level edge coverage. Buyer coverage is 215,218/215,221(99.999% after rounding), supplier coverage is 204,186/215,221(94.87%), category coverage is 164,691/215,221(76.52%), and at least one evidence edge is available for 215,202/215,221contracts (99.99%). These are structural coverage measures, not correctness rates.

### 5.2 PACS construction

PACS is designed to test combinations of closed-algebra primitives rather than broad world knowledge. Seven families cover: F1 spending and volume; F2 temporal change; F3 supplier concentration; F4 cross-buyer comparison; F5 overlap and exclusion; F6 relational composition; and F7 disclosure/compliance. L1 questions use an atomic or shallow computation, L2 questions combine two or three operations, and L3 questions use nested set membership, filtered comparisons, set algebra, or group composition.

Construction proceeds as follows. First, a family template samples anchors from the frozen record universe and instantiates a gold tree. Second, the tree must pass static validation, execute successfully in both implementations, produce agreeing answers, and satisfy a family-specific non-degeneracy rule. Third, supported variants are paired with out-of-grammar requests and scopes that produce a clean empty denotation. Fourth, the generator records a tree hash and a shape signature that removes literals while preserving operator

---

structure. Fifth, whole intent clusters—including their status variants—are assigned to development or test with a fixed seed, so closely related variants do not cross the split.

Each test intent has two evaluated surfaces. Channel A is a deterministic independent rendering of the semantics. Channel B is a separately naturalised question that must preserve required literals and pass a logic gate. The two channels share the gold program and oracle, allowing a paired robustness comparison without changing the underlying computation.

The sealing code enforces exact normalised question non-overlap with the training export, gold tree-hash non-overlap, and absence of each row labelled *unseen* from the set of training shape signatures. It also screens Channel B for trigram Jaccard similarity of at least 0.7 against every seventh training trigram set. The source documentation names five gates, but no separate G3 test is implemented and the near-duplicate screen is subsampled. We therefore call PACS a *mechanically screened compositional test*, not an exhaustive zero-leakage or maximum-compound-divergence split. The test-channel SHA-256 hashes and construction commands are listed in Appendix A.

### 5.3 Systems

The primary PACS comparison uses the same algebra description and one-question prompt for all systems:

- **Untuned 8B:** Qwen3-8B emits a tree or abstention without task-specific fine-tuning.
- **Cloud planner:** `grok-4-1-fast-non-reasoning` uses the same free-decoding interface and algebra contract.
- **Typed-program 8B:** the Qwen3-8B QLoRA adapter described in Section 4.5.

The main protocol uses temperature zero, a single free decode, no guided JSON grammar, and no reflection/repair. This preserves comparability with the cloud model. Only the specialised parser was evaluated on Channel B; Channel A therefore provides the fair three-system comparison, while A–B measures within-model surface robustness.

The secondary end-to-end comparison contains 13 categories with 20 cases each: ambiguous, no-result, and unsupported abstention; Boolean, bridge-join, categorical, comparison, count, factoid, minimum/maximum, set, sum, and top-*k* questions. We compare the full local runtime, the same Qwen3 stack without specialised adaptation, and the stored cloud-teacher run. These results are tied to frozen summaries under the external `outputs` artifact store; the exact model/run manifest is less complete than for PACS, so they are reported as secondary system evidence rather than the main benchmark.

### 5.4 Metrics and statistical reporting

The primary metric is strict accuracy from Equation 4. It credits an answerable item only when a valid tree executes and its typed answer matches the oracle; an unsupported item requires explicit abstention. The supplementary *safe accuracy* also credits a literal-faithful multi-answer or empty-result execution for eligible non-answerable cases. Because this metric mixes exact answers and operationally safe outcomes, it is relegated to Appendix D. Tree-valid rate measures only static validity and is not a correctness metric.

We report exact numerators and denominators. Parenthesised intervals in the main tables are 95% Wilson score intervals for a binomial proportion; they describe uncertainty on the fixed test items, not model-training variation. All headline model evaluations are single deterministic decodes, so they do not estimate sensitivity to checkpoint, prompt, provider, or hardware variation. Family macro accuracy is the unweighted mean across F1–F7; overall accuracy is the micro average across all 922 cases.

## 6 Results

Table 4: PACS strict execution accuracy. All Channel-A rows use the same 922 cases and a single temperature-zero free decode. “Valid tree” is reported for diagnosis only: explicit abstentions do not emit a tree, and validity does not imply correctness.

| System                        | Surface | Correct    | $n$ | Strict acc.  | 95% Wilson CI         | Valid tree |
|-------------------------------|---------|------------|-----|--------------|-----------------------|------------|
| Untuned Qwen3-8B              | A       | 332        | 922 | 36.01        | [32.97, 39.16]        | 77.77      |
| Cloud planner                 | A       | 450        | 922 | 48.81        | [45.59, 52.03]        | 52.28      |
| <b>Typed-program Qwen3-8B</b> | A       | <b>687</b> | 922 | <b>74.51</b> | <b>[71.60, 77.22]</b> | 71.58      |
| Typed-program Qwen3-8B        | B       | 675        | 922 | 73.21        | [70.26, 75.97]        | 70.50      |

Table 5: Strict accuracy by PACS family. Exact correct counts are given in parentheses. Surface B is available only for the specialised parser.

| Family ( $n$ ) |                              | Untuned A  | Cloud A           | Typed A            | Typed B           |
|----------------|------------------------------|------------|-------------------|--------------------|-------------------|
| F1             | Spending/volume (148)        | 41.89 (62) | 62.84 (93)        | <b>84.46 (125)</b> | 81.08 (120)       |
| F2             | Temporal change (149)        | 57.72 (86) | 72.48 (108)       | <b>83.22 (124)</b> | 81.88 (122)       |
| F3             | Supplier concentration (135) | 19.26 (26) | 51.85 (70)        | 51.11 (69)         | <b>54.07 (73)</b> |
| F4             | Cross-buyer comparison (147) | 39.46 (58) | 44.22 (65)        | <b>81.63 (120)</b> | 79.59 (117)       |
| F5             | Overlap/exclusion (127)      | 32.28 (41) | 33.07 (42)        | <b>94.49 (120)</b> | 93.70 (119)       |
| F6             | Relational composition (149) | 28.19 (42) | 28.86 (43)        | <b>67.79 (101)</b> | 65.10 (97)        |
| F7             | Disclosure/compliance (67)   | 25.37 (17) | <b>43.28 (29)</b> | 41.79 (28)         | 40.30 (27)        |
| Macro family   |                              | 34.88      | 48.09             | <b>72.07</b>       | 70.82             |

### 6.1 Typed-program supervision more than doubles strict accuracy

Table 4 answers RQ1. The untuned Qwen3-8B model solves 332/922cases (36.01%). The cloud planner improves this to 450/922(48.81%). The typed-program model solves 687/922(74.51%), an absolute gain of 38.50 percentage points over the same base model and 25.70 points over the cloud planner. The Wilson intervals do not overlap either baseline interval.

Tree-valid rate alone ranks the untuned base above the specialised model, despite a 38.50-point gap in strict accuracy. This is direct evidence for the distinction in Table 1: a syntactically and type-valid program may still reverse roles, select the wrong field, or express the wrong scope. Conversely, an explicit abstention is a scoreable final action without a tree.

### 6.2 Surface naturalisation causes a small aggregate drop

For RQ2, the specialised model falls from 74.51% on Channel A to 73.21% on Channel B, a difference of 1.30 percentage points. Macro-family accuracy falls from 72.07% to 70.82%. This suggests that the parser is not merely reproducing one fixed wording, but the result should not be interpreted as unrestricted linguistic robustness: both surfaces preserve the same literals and are generated under explicit logic gates.

The largest A–B family decline is 3.38 points on relational composition (F6); supplier concentration (F3) instead improves by 2.96 points. A paired item-level significance test is left out of the present draft because the intended strict Boolean-matching fix must first be applied uniformly to every scorer and the paired decisions regenerated from the frozen outputs.

### 6.3 Improvements are concentrated in compositional set operations

Table 5 answers RQ3. The specialised parser is strongest on overlap and exclusion (F5, 94.49%) and improves the base by more than 60 points. It also substantially improves cross-buyer comparison (F4) and relational composition (F6). Supplier concentration (F3) and disclosure/compliance (F7) remain near 50% and 42%, respectively; on both families the cloud model is competitive with or slightly above the specialised model. These failures are obscured by the micro average, particularly because F7 contains only 67 cases.

Table 6: Secondary full-runtime result on a balanced frozen artifact (13 categories, 20 cases each). The incomplete external run manifest makes this weaker reproducibility evidence than Table 4.

| System                    | Correct    | $n$ | Accuracy     | 95% Wilson CI         |
|---------------------------|------------|-----|--------------|-----------------------|
| Untuned local runtime     | 183        | 260 | 70.38        | [64.57, 75.60]        |
| Cloud-teacher runtime     | 192        | 260 | 73.85        | [68.18, 78.81]        |
| <b>Full local runtime</b> | <b>224</b> | 260 | <b>86.15</b> | <b>[81.43, 89.83]</b> |

Depth diagnostics are not monotonic. The typed parser obtains 92.11% at L1, 61.06% at L2, and 73.36% at L3. The lower L2 value is caused by the distribution of families and status types rather than proving that two-step programs are intrinsically harder than L3. Declared seen and unseen shapes score 74.01% and 74.95%, respectively. This near equality is encouraging, but it is not by itself proof of CFQ-style compositional generalisation because the split lacks an atom/compound-divergence construction and its near-duplicate screen is subsampled.

#### 6.4 The full runtime improves on the balanced system snapshot

Table 6 answers RQ4 on the separate 260-item snapshot. The full local runtime solves 224/260 cases (86.15%), compared with 183/260 (70.38%) for the untuned local system and 192/260 (73.85%) for the stored cloud-teacher run. The largest category differences relative to the untuned system occur on bridge joins (14/20 versus 2/20), comparisons (17/20 versus 7/20), and categorical questions (13/20 versus 8/20). All three systems score 20/20 on the snapshot’s sum category; because the underlying data contain unnormalised non-GBP additive rows, we do not interpret that category as evidence of currency-correct aggregation.

#### 6.5 Qualitative bridge case

One PACS F6–L3 case asks for the buyers reached by first finding the lead suppliers of an anchor buyer, following those supplier values to other notices, and excluding the exact anchor string. The untuned model filters `supplier_name` by the buyer literal and projects `contract_node_id`; its well-typed set difference executes to the empty set. The specialised parser instead emits Equation 7 and returns the 17 stored buyer-name strings in the oracle. The output includes surface variants such as “EDUCATION AUTHORITY”, “Education Authority NI”, and “the Education Authority”, which is useful evidence of both the compositional success and the unresolved entity-name limitation.

This case belongs to the direct typed-program protocol. It demonstrates semantic parsing and deterministic set execution, but it is not evidence that the deployment understander, conditional planner, or release checks ran for that question. A Figure-1 end-to-end example must therefore be exported from one full runtime trace rather than reusing this PACS row.

## 7 Analysis and Discussion

### 7.1 Where the aggregate gain comes from

The specialised parser’s 687 strict successes consist of 549 correct executed answers and 138 correct explicit abstentions. In contrast, the untuned model produces only 291 correct answers and 41 correct abstentions; it answers 114 out-of-grammar cases and emits 104 invalid trees. Typed-program supervision therefore changes both semantic coverage and the model’s willingness to use the unsupported action. It is not merely a syntax repair: the base model already emits a high fraction of type-valid trees, yet many encode the wrong role, projection, or composition.

The cloud planner produces 324 correct answers and 126 correct abstentions, but also 161 invalid trees, 57 unparseable outputs, and 25 API errors. Counting service errors as failures is appropriate for an end-to-end single-run comparison, but it mixes model capability with provider availability. A future controlled comparison should repeat the run, report provider failures separately, and keep the same retry budget for every system.

---

## 7.2 Strict and safe outcomes answer different questions

The specialised Channel-A run contains 73 literal-faithful empty-result executions. The strict metric does not credit them because the benchmark requires an explicit abstention for those non-answerable controls. The supplementary safe metric does, raising the score from 74.51% to 82.43%. This gap is informative but should not be presented as a second answer-accuracy number: it measures whether the model avoided an unsupported factual answer under a broader operational policy. The full outcome counts and definition appear in Appendix D.

This distinction also exposes a product decision. A user asking for a count over an empty scope might reasonably prefer “0”, while a missing-operator question requires an explicit statement that the operation is unsupported. A production interface should make these policies visible rather than merge both into a generic refusal label.

## 7.3 Composition is not equivalent to depth

The non-monotonic L1/L2/L3 results show that depth is an incomplete difficulty measure. An L3 set difference can be structurally regular and well represented in supervision, whereas an L2 grouped concentration query may require a rarer operator and harder return type. The family results support reporting a two-dimensional family-by-depth matrix, with cell counts beside percentages, rather than a single line labelled “accuracy versus complexity.” The final figure brief in the repository specifies this diagnostic and retains the exact table in the appendix.

Declared unseen shapes also do not imply unseen primitives. PACS records whether an exact shape signature occurs in the training export, but it does not calculate atom and compound divergence as CFQ does. The 74.95% unseen-shape score therefore supports generalisation to the benchmark’s declared structural holdout; it does not establish systematic compositional generalisation in the stronger formal sense.

## 7.4 What the full-runtime result can and cannot attribute

The 86.15% system result is consistent with the intended benefit of specialised planning inside an execute-and-check runtime. Category gains on bridge, comparison, and set questions align with the PACS family improvements. Nevertheless, the comparison does not isolate the understander, router, deterministic compiler, graph planner, verifier, or repair loop. A causal component claim would require compiler-only, always-planner, conditional-routing, no-verifier, and no-repair variants on the same snapshot and budget. We therefore report the observed full stack difference and avoid assigning it to any single component.

The runtime’s strongest methodological property is instead inspectability. A failure can be localised to an unparseable intent, routing veto, graph inconsistency, grounding error, preflight failure, empty intermediate set, postflight defect, or evidence decision. This is a practical improvement over direct answer generation even when the final accuracy is unchanged. It should be called *auditable* rather than *fully interpretable*: the learned understander and planner do not provide a complete causal explanation of their own decisions.

## 7.5 Data quality remains part of model quality

The bridge example returns 17 distinct stored names, but multiple strings appear to refer to the same organisational family. An exact set match can therefore be correct under the benchmark and still be inconvenient for a user who expects resolved legal entities. Similarly, a correct program over an unnormalised currency column can return a reproducible but semantically mislabelled total. These are not peripheral data-engineering issues: they define the denotation against which the model is trained and scored. Future versions should version the entity and currency semantics together with the benchmark, rather than updating the KG under an unchanged test label.



---

## 8 Limitations

**Finite and publisher-dependent data.** The graph is a 2022–2026 snapshot selected from published OCDS releases. Exhaustive execution means exhaustive over the declared query-record universe, not exhaustive over all procurement activity. Late, incorrect, or absent disclosures remain absent from the answer. Keeping only the latest release per OCID also discards event-history detail.

**Entity identity.** Entity resolution is conservative and stored query fields can contain unresolved aliases. A value-set answer is therefore a set of strings, not necessarily a set of unique legal entities. The current source code includes corrective handling for role-safe publication-system aliases, blank-name singletons, and globally ambiguous alias candidates, but the frozen graph used for the reported experiments was built before all of these patches. Results must remain bound to the old graph hash until the pipeline is rebuilt and the benchmark is versioned; silently substituting a new graph would change the task.

**Currencies and monetary semantics.** Of 176,002 rows marked additive in the frozen record universe, 1,059 have a non-GBP currency. The algebra and full runtime require an additivity guard for intended sums, but neither performs currency conversion and the generic algebra executor does not hard-enforce a currency filter. Some prompts nevertheless describe totals as GBP. Monetary results can thus mix currencies, and the sum-related benchmark results should not be interpreted as currency-correct financial analysis. A new benchmark version should add currency to the value type or reject mixed scopes.

**Synthetic supervision and split isolation.** Program supervision is generated from templates and sampled anchors. The current training export contains exact duplicates and two exact train/validation overlaps, so its validation loss is not independent evidence. PACS removes exact question and tree-hash collisions and controls declared shape exposure, but its near-duplicate screen compares Channel B against only every seventh training trigram set, and the documented G3 gate is not separately implemented. PACS therefore measures a useful, mechanically screened structural holdout, not a proof of zero leakage or natural user distribution.

**Historical comparison semantics.** Several data-generation and scoring helpers used Boolean coercion that can equate a Boolean with a non-Boolean truthy or falsy value. A strict audit found no such mismatch in the stored PACS typed-program A run, but the helper remains a risk for legacy datasets and future generation. Every scorer should use a shared type-strict comparator before paired significance tests or new headline results are produced. We exclude legacy result sets known to be affected.

**Single runs and incomplete ablations.** The headline evaluations use one temperature-zero decode per item. Wilson intervals quantify test-item uncertainty but not training-seed, checkpoint, prompt, provider, or serving variability. We also lack a complete same-budget ablation of compiler-only, always-planner, conditional routing, verification, and bounded repair. Consequently, the end-to-end result supports the full configuration but not a causal contribution for each module.

**Verifier scope.** Static and runtime checks can reject invalid objects, unsupported fields, malformed executions, and insufficient implemented evidence. They cannot prove that a valid program represents the question, that an accepted answer is complete in the world, or that a cited record is itself correct. Runtime-verified rate must therefore remain separate from oracle match and should not be described as hallucination-free or guaranteed semantic correctness.

**Artifact portability.** The PACS inputs and per-item outputs are stored in the repository, but several end-to-end model adapters and evaluation summaries are reached through a machine-specific absolute `outputs` link. The large 2,285-item final summaries also omit category/configuration metadata. We consequently use the self-contained PACS artifacts as the primary evidence and label the 260-item runtime result as a frozen artifact. A release should replace the absolute link with content-addressed manifests and publish model, prompt, environment, and evaluator hashes.

---

**Domain and user evaluation.** The study is English-language and UK-procurement specific. It does not establish transfer to other jurisdictions, schemas, or open-domain KGQA. Nor does exact program accuracy measure whether procurement practitioners understand or trust the returned evidence. A future study should evaluate task completion, error detection, and the usefulness of abstention explanations with domain users.

## 9 Conclusion

This work studies public-procurement question answering as typed semantic parsing followed by deterministic execution, rather than as direct factual text generation. A provenance-preserving graph supplies a finite query universe; a closed algebra represents set, bridge, aggregate, comparison, and ranking computations; and a full runtime adds structured understanding, conditional planning, grounding, evidence checks, bounded replanning, and controlled non-answering. On PACS, typed-program adaptation raises strict accuracy from 36.01% to 74.51% and retains 73.21% under independent surface naturalisation. A separate system snapshot shows an improvement from 70.38% to 86.15% for the full local runtime.

The audits are as important as the gains. Type validity is not semantic correctness; runtime verification is not an oracle; exhaustive execution is only exhaustive within a frozen snapshot; and unresolved aliases and currencies can change what an otherwise correct program denotes. The next defensible step is therefore not a larger headline experiment, but a versioned rebuild that enforces strict answer types and currency semantics, publishes portable manifests, and evaluates routing, verification, and repair with controlled ablations. This would turn the current executable prototype into a reproducible study of when typed, evidence-linked question answering succeeds and when it should decline to answer.

## Broader Impact and Ethics

The source records describe public contracting activity, but apparently simple system outputs can still affect the reputation of named organisations. The pipeline is intended for data exploration and audit support, not for automated allegations, supplier scoring, or procurement decisions. An answer should be accompanied by its data scope, provenance, and limitations; missing records, unresolved aliases, and unnormalised currencies must not be silently converted into claims about misconduct. Public availability of a record also does not remove the need to review licensing, personal-data fields, and downstream use before redistribution.

## References

- Shulin Cao, Jiaxin Shi, Liangming Pan, Lunyu Nie, Yutong Xiang, Lei Hou, Juanzi Li, Bin He, and Hanwang Zhang. KQA pro: A dataset with explicit compositional programs for complex question answering over knowledge base. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6101–6119, 2022. doi: 10.18653/v1/2022.acl-long.422. URL <https://aclanthology.org/2022.acl-long.422/>.
- Shuang Chen, Qian Liu, Zhiwei Yu, Chin-Yew Lin, Jian-Guang Lou, and Feng Jiang. ReTraCk: A flexible and efficient framework for knowledge base question answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, pp. 325–336, 2021. doi: 10.18653/v1/2021.acl-demo.39. URL <https://aclanthology.org/2021.acl-demo.39/>.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL [https://proceedings.neurips.cc/paper\\_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html).
- Wangzhen Guo, Linyin Luo, Hanjiang Lai, and Jian Yin. From parse-execute to parse-execute-refine: Improving semantic parser for complex question answering over knowledge base. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 11771–11780, 2023. doi: 10.18653/v1/2023.emnlp-main.720. URL <https://aclanthology.org/2023.emnlp-main.720/>.

- 
- Xiang Huang, Sitao Cheng, Shanshan Huang, Jiayu Shen, Yong Xu, Chaoyun Zhang, and Yuzhong Qu. QueryAgent: A reliable and efficient reasoning framework with environmental feedback based self-correction. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5014–5035, 2024. doi: 10.18653/v1/2024.acl-long.274. URL <https://aclanthology.org/2024.acl-long.274/>.
- Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Xin Zhao, and Ji-Rong Wen. StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 9237–9251, 2023a. doi: 10.18653/v1/2023.emnlp-main.574. URL <https://aclanthology.org/2023.emnlp-main.574/>.
- Jinhao Jiang, Kun Zhou, Xin Zhao, and Ji-Rong Wen. UniKGQA: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph. In *International Conference on Learning Representations*, 2023b. URL <https://openreview.net/forum?id=Z63RvyAZ2Vh>.
- Daniel Keysers, Nathanael Sch"arli, Nathan Scales, Hylke Buisman, Daniel Furrer, Sergii Kashubin, Nikola Momchev, Danila Sinopalnikov, Lukasz Stafiniak, Tibor Tihon, Dmitry Tsarkov, Xiao Wang, Marc van Zee, and Olivier Bousquet. Measuring compositional generalization: A comprehensive method on realistic data. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=SygcCnNKwr>.
- Najoung Kim and Tal Linzen. COGS: A compositional generalization challenge based on semantic interpretation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 9087–9105, 2020. doi: 10.18653/v1/2020.emnlp-main.731. URL <https://aclanthology.org/2020.emnlp-main.731/>.
- Chen Liang, Jonathan Berant, Quoc Le, Kenneth D. Forbus, and Ni Lao. Neural symbolic machines: Learning semantic parsers on Freebase with weak supervision. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 23–33, 2017. doi: 10.18653/v1/P17-1003. URL <https://aclanthology.org/P17-1003/>.
- Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations*, 2024. URL [https://proceedings.iclr.cc/paper\\_files/paper/2024/hash/3e2aeb66481dd63a32421bf032b70384-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2024/hash/3e2aeb66481dd63a32421bf032b70384-Abstract-Conference.html).
- Open Contracting Partnership. Open contracting data standard 1.1.5: Reference. Online documentation, 2026. URL <https://standard.open-contracting.org/latest/en/schema/>. Accessed 12 August 2026.
- Inbar Oren, Jonathan Herzig, Nitish Gupta, Matt Gardner, and Jonathan Berant. Improving compositional generalization in semantic parsing. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 2482–2495, 2020. doi: 10.18653/v1/2020.findings-emnlp.225. URL <https://aclanthology.org/2020.findings-emnlp.225/>.
- Mayur Patidar, Prayushi Faldu, Avinash Singh, Lovekesh Vig, Indrajit Bhattacharya, and Mausam. Do I have the knowledge to answer? investigating answerability of knowledge base questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10341–10357, 2023. doi: 10.18653/v1/2023.acl-long.576. URL <https://aclanthology.org/2023.acl-long.576/>.
- Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 9895–9901, 2021. doi: 10.18653/v1/2021.emnlp-main.779. URL <https://aclanthology.org/2021.emnlp-main.779/>.
- Peter Shaw, Ming-Wei Chang, Panupong Pasupat, and Kristina Toutanova. Compositional generalization and natural language variation: Can a semantic parsing approach handle both? In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 922–938, 2021. doi: 10.18653/v1/2021.acl-long.75. URL <https://aclanthology.org/2021.acl-long.75/>.

- 
- Yiheng Shu, Zhiwei Yu, Yuhang Li, Björn F. Karlsson, Tingting Ma, Yuzhong Qu, and Chin-Yew Lin. TIARA: Multi-grained retrieval for robust question answering over large knowledge base. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 8108–8121, 2022. doi: 10.18653/v1/2022.emnlp-main.555. URL <https://aclanthology.org/2022.emnlp-main.555/>.
- Ahmet Soylu, Oscar Corcho, Brian Elvesaeter, Carlos Badenes-Olmedo, Tom Blount, Fernando Yedro Martínez, Mateja Kovacic, Marco Posinkovic, Ian Makgill, Chris Taggart, Elena Simperl, Tomasz C. Lech, Dumitru Roman, and Jens Lehmann. Theybuyforyou platform and knowledge graph: Expanding horizons in public procurement with open linked data. *Semantic Web*, 13(2):265–291, 2022. doi: 10.3233/SW-210442. URL <https://doi.org/10.3233/SW-210442>.
- Haitian Sun, Bhuwan Dhingra, Manzil Zaheer, Kathryn Mazaitis, Ruslan Salakhutdinov, and William Cohen. Open domain question answering using early fusion of knowledge bases and text. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 4231–4242, 2018. doi: 10.18653/v1/D18-1455. URL <https://aclanthology.org/D18-1455/>.
- Haitian Sun, Tania Bedrax-Weiss, and William Cohen. PullNet: Open domain question answering with iterative retrieval on knowledge bases and text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 2380–2390, 2019. doi: 10.18653/v1/D19-1242. URL <https://aclanthology.org/D19-1242/>.
- Bailin Wang, Mirella Lapata, and Ivan Titov. Learning from executions for semantic parsing. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2747–2759, 2021. doi: 10.18653/v1/2021.naacl-main.219. URL <https://aclanthology.org/2021.naacl-main.219/>.
- Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. Chain-of-table: Evolving tables in the reasoning chain for table understanding. In *International Conference on Learning Representations*, 2024. URL [https://proceedings.iclr.cc/paper\\_files/paper/2024/hash/f53fd88a4340063ecd258c0ae9948b40-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2024/hash/f53fd88a4340063ecd258c0ae9948b40-Abstract-Conference.html).
- Tomer Wolfson, Mor Geva, Ankit Gupta, Matt Gardner, Yoav Goldberg, Daniel Deutch, and Jonathan Berant. Break it down: A question understanding benchmark. *Transactions of the Association for Computational Linguistics*, 8:183–198, 2020. doi: 10.1162/tacl\_a\_00309. URL <https://aclanthology.org/2020.tacl-1.13/>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.

## A Reproducibility Snapshot

### A.1 Code and artifact state

The repository base commit inspected for this draft is `d15ec0a3bd5b5f295f9828e44067094b73ab2cd7`. The working tree also contains uncommitted correctness patches and staged data artifacts; the commit alone is therefore not an exact reproduction capsule. Before submission, the authors must create a clean tagged revision and record the patch, Python environment lock, CUDA/driver versions, model revisions, and command lines. No new model training or batch inference was run to prepare this draft.

Per-item PACS predictions and summaries are stored under `data/qa/compose_probe_v1/`. The primary summaries are `summary_pacstest_base_a.json`, `summary_pacstest_teacher_a.json`, `summary_pacstest_v3_a.json`, and `summary_pacstest_v3_b.json`. The corresponding JSON Lines files retain the raw model response, parsed tree, execution answer, oracle, outcome, and correctness flag.

Table 7: Content hashes for the primary frozen inputs.

| Repository path                                              | SHA-256                                                              |
|--------------------------------------------------------------|----------------------------------------------------------------------|
| data/qa/pacs_v1/test_channel_a.jsonl                         | 1869558ad2764b92888df47b093dfc2f<br>f693d2fd6dd6544f1fa3734d017cf2bb |
| data/qa/pacs_v1/test_channel_b.jsonl                         | 36100c4bf7b34ef6fe1d72c62d33a62<br>d069501aad97c0e28a172345a901339b  |
| data/training/llamafactory_compose_v3/compose_sft_train.json | a80945184aeb9531321b26ef57286656<br>8707eeba10b7dc56ec34f8e35f00a20d |
| data/training/llamafactory_compose_v3/compose_sft_val.json   | 7cba3d43610a90216e5d7655319178df<br>3a66f1728d1133b704808b0a8ba2a899 |
| data/kg/nodes/contract_nodes.parquet                         | 246caad93e1c8b184d3714a280190efb<br>3c238f4cab73cd3bf1f36b38de307bf1 |
| data/kg/nodes/org_nodes.parquet                              | 42f9542833275ea7a57feb2cc4f4e64<br>a507513f35b2e420a449d938b68b867af |

The 260-item runtime summaries are under `outputs/eval/matrix_v2/fully-local-qwen/`, `.../cicada-qwen3-zeroshot/`, and `.../teacher_r3/`. These paths contain an internal historical label and currently resolve through a machine-specific absolute symlink; that label is not used as the method’s paper name.

## A.2 Pipeline entry points

A full deterministic data build follows this order:

```
.venv/bin/python -B pipelines/01_ingest.py
.venv/bin/python -B pipelines/02_er_phase1.py
.venv/bin/python -B pipelines/03_er_phase2.py
.venv/bin/python -B pipelines/04_extract.py
.venv/bin/python -B pipelines/40_build_kg.py
.venv/bin/python -B pipelines/41_validate_kg.py
```

Partial year/sample runs require explicit alternate outputs in the current code and must never be used to overwrite the canonical artifact. Exact raw-data acquisition commands and licences remain to be added to the release manifest.

Program supervision is built with `scripts/build_compose_train.py`, exported with the compose export script, and trained with `configs/training/qwen3_8b_compose_sft_v3_qlora.yaml`. The stored training summary reports train loss 0.0171394, evaluation loss 0.0096943, and runtime 8270.93 seconds. These losses are diagnostics only because the exported split is not fully deduplicated.

PACS generation uses, in order, `scripts/pacs/generate.py`, `render_channels.py`, `surface_independent.py`, `naturalize.py`, and `seal.py`. Evaluation uses `scripts/run_compose_probe_eval.py`. A release command must record the serving endpoint, exact model revision, API date/version, prompt hash, maximum tokens, concurrency, temperature, guided-decoding flag, reflection budget, and scorer revision.

## A.3 Checklist before final submission

The final artifact should contain: (i) a clean source tag and dirty-tree check; (ii) raw-data and KG manifests; (iii) model/adaptor hashes or durable release links; (iv) exact prompts and JSON schemas; (v) strict scorer unit tests covering Boolean versus numeric values; (vi) route, repair, and failure counts; (vii) random seeds and repeated-run policy; (viii) hardware and software versions; and (ix) a one-command script that regenerates every table from JSON/Parquet without copying numbers by hand.

## B Complete Algebra Contract

Base predicates are `eq`, `in`, `contains`, `exists`, `gte`, and `lte`. `not` negates one predicate, `any` disjoins two or more predicates, and `in_expr` evaluates a `Valueschild` and applies membership or negated membership to a

Table 8: Complete expression-node inventory in the current validator.

| Node                      | Input type(s)                  | Output         |
|---------------------------|--------------------------------|----------------|
| <code>filter</code>       | predicates over $U_G$          | Records        |
| <code>values</code>       | Records, field                 | Values         |
| <code>count</code>        | Records                        | Number         |
| <code>size</code>         | Values                         | Number         |
| <code>sum</code>          | Records, field                 | Number         |
| <code>exists</code>       | Records                        | Bool           |
| <code>select</code>       | Records, field                 | Value          |
| <code>extreme_rows</code> | Records, field, argmin/argmax  | Records        |
| <code>extreme</code>      | Records, field, argmin/argmax  | Value          |
| <code>groupby</code>      | Records, key, count/sum metric | Groups         |
| <code>argext</code>       | Groups, argmin/argmax          | Value          |
| <code>top</code>          | Groups, $1 \leq k \leq 50$     | Ranking        |
| <code>num</code>          | numeric non-Boolean literal    | Number         |
| <code>combine</code>      | Number, Number                 | Number or Bool |
| <code>vcompare</code>     | Value, literal                 | Bool           |
| <code>setop</code>        | Values, Values                 | Values         |
| <code>gcombine</code>     | Groups, Groups                 | Groups         |
| <code>keys_where</code>   | Groups, comparator, threshold  | Values         |

field. Numeric `combine` operators are `diff`, `ratio`, and `add`; Boolean comparisons are `gt`, `lt`, `ge`, `le`, and `eq`. Set operators are union, intersection, and difference.

The validator enforces compositional types, required arguments, a maximum depth of 16, a maximum of 64 nodes, and  $k \leq 50$ . It does not restrict every field to the runtime schema and it does not make the additivity or currency guard intrinsic to `sum`. Those are separate domain contracts. Runtime failure modes include unknown fields, non-finite/non-numeric sums, no result, and multiple distinct values for a scalar selection. For `extreme_rows`, all tied rows at the extremum are retained; downstream scalar selection therefore fails on unresolved ties instead of choosing an arbitrary row.

Counts and grouped counts deduplicate contract identifiers. Projections return sorted distinct non-empty strings. Grouped keys exclude null and empty strings. Numeric sums use decimal accumulation before conversion to the JSON answer representation. Ranking ties are broken deterministically by the key string after the numeric score.

## C PACS Construction Details

Table 9: PACS capability families and intended composition at each level.

| ID | Capability             | Typical L1–L3 progression                                                                                  |
|----|------------------------|------------------------------------------------------------------------------------------------------------|
| F1 | Spending/volume        | Filtered count or additive sum; combined scopes; nested supplier-conditioned scope.                        |
| F2 | Temporal change        | Existence/count by year; comparison or difference; grouped temporal extreme.                               |
| F3 | Supplier concentration | Supplier count; grouped top supplier; buyer-conditioned supplier grouping.                                 |
| F4 | Cross-buyer comparison | Buyer scope; same-category compare/difference; differently filtered two-sided comparison.                  |
| F5 | Overlap/exclusion      | Supplier set; union/intersection/difference; record count conditioned on a derived set.                    |
| F6 | Relational composition | Buyers of supplier; year-filtered buyer set; buyer–supplier– buyer bridge with anchor exclusion.           |
| F7 | Disclosure/compliance  | Missing-field count; buyers with at least one missing field; grouped threshold/perfect-coverage condition. |

The 922-case test contains 148/149/135/147/127/149/67 examples from F1–F7. Depth counts are 279 L1, 339 L2, and 304 L3. Shape exposure is 431 seen and 491 declared unseen. Operational status counts are 694 answerable, 138 unsupported, and 90 no-result cases. Because status variants travel with their base intent cluster, they do not cross the development/test split.

The sealing implementation uses seed 20260719. It drops an entire cluster if any of its three surface candidates exactly collides with a normalised training instruction, relabels status variants from their actual tree shape, and checks the exact question, tree hash, and unseen-shape conditions. For the naturalised channel it computes

character-trigram Jaccard similarity against `train_tris[:,7]`; this performance shortcut is disclosed because it weakens an otherwise all-pairs interpretation of the gate. The console message currently says that G3 passes even though no separate G3 counter is evaluated. The paper wording follows code, not that message.

The audit sample selects up to five naturalised instances per family–depth cell and additional status examples with the same seed. Before release, a human audit should record literal preservation, semantic equivalence, answerability, and whether any generated wording is unnatural or discloses a template artifact. The current repository contains the sample but not a complete signed human-audit report.

## D Full Diagnostic Results

Table 10: PACS outcome summary. Safe accuracy adds faithful multi-answer/empty-result outcomes to strict correctness; it is not a second exact-answer metric.

| System/surface | Strict | Safe  | Abstain | Answered | Faithful empty | Invalid tree |
|----------------|--------|-------|---------|----------|----------------|--------------|
| Untuned A      | 36.01  | 42.41 | 41      | 603      | 59             | 104          |
| Cloud A        | 48.81  | 54.99 | 130     | 480      | 57             | 161          |
| Typed A        | 74.51  | 82.43 | 138     | 655      | 73             | 51           |
| Typed B        | 73.21  | 81.13 | 138     | 645      | 73             | 60           |

The cloud row additionally contains 25 API errors, 57 unparseable outputs, and 10 evaluation failures. The typed-B row contains one evaluation failure. Outcome categories are mutually exclusive within each stored evaluation file, but an outcome such as *answered* can be correct or incorrect.

Table 11: Strict accuracy by depth and declared shape exposure.

| Slice                 | <i>n</i> | Untuned A | Cloud A | Typed A | Typed B |
|-----------------------|----------|-----------|---------|---------|---------|
| L1                    | 279      | 50.54     | 61.29   | 92.11   | 90.32   |
| L2                    | 339      | 27.14     | 46.90   | 61.06   | 59.29   |
| L3                    | 304      | 32.57     | 39.47   | 73.36   | 73.03   |
| Seen shape            | 431      | 46.64     | 61.48   | 74.01   | 72.39   |
| Declared unseen shape | 491      | 26.68     | 37.68   | 74.95   | 73.93   |

Table 12: Correct cases out of 20 by category in the frozen full-runtime snapshot.

| Category               | Untuned local | Cloud teacher | Full local |
|------------------------|---------------|---------------|------------|
| Ambiguous abstention   | 19            | 19            | 19         |
| No-result abstention   | 20            | 20            | 20         |
| Unsupported abstention | 19            | 19            | 19         |
| Boolean                | 14            | 13            | 14         |
| Bridge join            | 2             | 0             | 14         |
| Categorical            | 8             | 11            | 13         |
| Comparison             | 7             | 11            | 17         |
| Count                  | 17            | 20            | 20         |
| Factoid                | 17            | 16            | 15         |
| Minimum/maximum        | 15            | 16            | 20         |
| Set                    | 11            | 13            | 15         |
| Sum                    | 20            | 20            | 20         |
| Top- <i>k</i>          | 14            | 14            | 18         |
| Overall                | 183           | 192           | 224        |

The repository also contains a 2,285-item final-test summary for several systems. We do not use it as a main result because its summary lacks category/configuration metadata and its adapters are not portable from the current machine-specific output store. Preserving an attractive number without a complete protocol would weaken rather than strengthen reproducibility.

## E Trace and Output Examples



---

## E.1 Direct compositional success and base failure

For PACS item PACS::F6:L3:f6\_other\_buyers\_via\_suppliers:0011#a, the question asks which buyer-name values are connected through the lead suppliers of an anchor buyer, excluding the exact anchor string. The untuned model emits the following compressed computation:

```
VALUES contract_node_id [supplier_name = anchor]
  DIFFERENCE
VALUES contract_node_id [buyer_name = anchor]
```

It reverses the role in the first filter and projects contract IDs, yielding the empty set. The typed-program model emits:

```
S = VALUES supplier_name [buyer_name = anchor]
B = VALUES buyer_name     [supplier_name IN_EXPR S]
A = B DIFFERENCE VALUES buyer_name [buyer_name = anchor]
```

The deterministic evaluator returns the 17 stored strings in the oracle. Representative values are “Ards and North Down Borough Council”, “Leeds City Council”, and “NI Water and its subsidiaries”. The output also contains organisation-name variants; it must be captioned as 17 stored buyer-name values, not 17 unique organisations.

## E.2 Unsupported sibling

A separate F6–L3 control asks, “What is the average number of buyers per supplier in this scope?” Average is outside the closed algebra. The specialised model returns an explicit abstention object with the reason that the operation is not expressible. This is an independent test item, not a failure branch of the answerable bridge question.

## E.3 Full-runtime figure slot

The final introduction figure requires one newly exported, current-code trace satisfying all of the following conditions:

- a live structured-understander artifact, including answer signature and source/bridge/ target roles;
- the actual router decision and either deterministic compiler output or Step-2 graph planner output (never both as executed branches);
- compiled variables and dependencies; grounded constraints; source, bridge, and target output sizes; and two or three values that can be traced to the same execution;
- evidence identifiers, preflight/postflight checks, answer-sanity and evidence verdict, and the final answer card; and
- a trace ID, model revisions, graph hash, timestamp, and offline oracle result shown only as an evaluation annotation.

Existing PACS rows do not satisfy these requirements because their protocol is a direct tree decode followed by offline scoring. Existing historical teacher traces may help select a question, but they should not be presented as a current full-runtime execution. Exporting one such trace is a lightweight single-case reproducibility task and does not require rerunning a batch experiment.